IMU-Based Head Tracking for Camera Control in a Virtual Panoramic Environment

Keyuan Zhang
KTH Royal Institute of Technology
keyuanz@kth.se

Abstract

This project investigates the use of a low-cost MPU6050 inertial measurement unit together with an Arduino Nano for real-time camera control in a virtual environment. Angular velocity measurements from the gyroscope are transmitted through serial communication to a Processing application, where they are integrated into yaw and pitch angles and mapped to the orientation of a virtual camera.

The project focuses on the complete interaction pipeline from physical motion sensing to real-time graphics, together with practical issues such as sensor noise, gyroscope drift, smoothing, and responsiveness.

A panoramic virtual environment recorded using personal smartphone is used as a proof-of-concept scene for evaluating the head-tracking interaction.

1 Introduction

Head tracking is commonly used in virtual reality, simulation, and driving applications to provide more natural control of the user's viewpoint. Commercial head-tracking solutions like the one done by Tobii Gaming which supports Euro Truck Simulator 2 [1] can provide high accuracy, but they are built using expensive dedicated hardware.

The objective of this project is to investigate whether a low-cost MPU6050 inertial measurement unit can be used as an alternative input device for controlling the user camera position in real time.

The MPU6050 is connected to an Arduino Nano development board, which acquires gyroscope measurements and transfers them to a Processing application through serial communication. The measured angular velocities are processed and mapped to the yaw and pitch motion of a virtual camera.

The main challenges considered in the project are sensor noise, gyroscope drift, real-time responsiveness, and the mapping between physical head movement and virtual camera orientation.

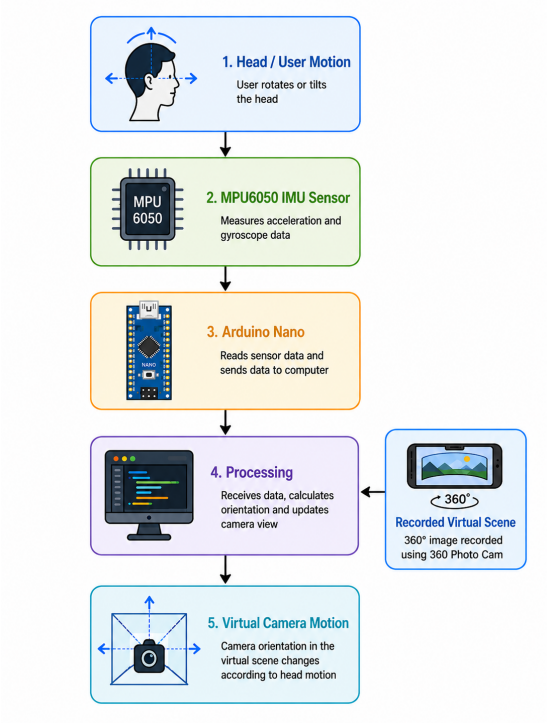

Figure 1: Overview of the IMU head-tracking system.

2 System Design

2.1 System Architecture

The complete system consists of an MPU6050 IMU, an Arduino Nano, serial communication, and a Processing-based virtual environment.

The data flow of the system can be seen in Figure 1. The MPU6050 gyroscopic sensor is connected with Arduino Nano to read in the user's motion then transmit the data through serial communication library to Processing. Then use the example recorded virtual scene (using phone app "360 Photo Cam") as testing environment to see how the view angle changes.

2.2 Hardware

The hardware system consists mainly of an Arduino Nano and an MPU6050 six-axis inertial measurement unit. The MPU6050 includes a three-axis gyroscope and a three-axis accelerometer.

In the current implementation, the gyroscope measurements are primarily used for controlling camera orientation in the virtual 360-degree picture.

The MPU6050 communicates with the Arduino using the I^2C protocol. The Arduino subsequently communicates with the computer through USB serial communication.

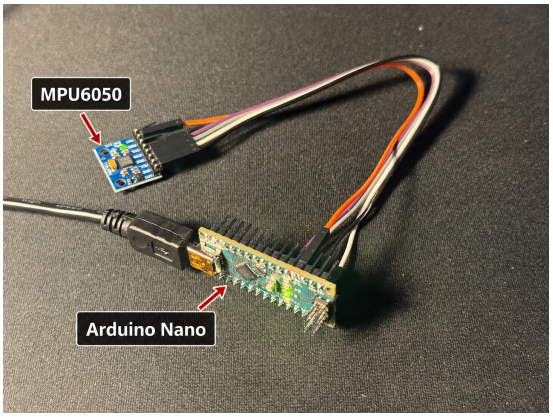

Figure 2: Arduino Nano and MPU6050 hardware setup.

2.3 Software

The software implementation is divided into three segments:

- Arduino firmware for sensor communication and data acquisition.
- Processing software for serial communication, orientation estimation, and virtual camera rendering.
- Phone app for recording the panoramic 360-degree picture

3 Implementation

3.1 MPU6050 Data Acquisition

The MPU6050 gyroscope produces raw integer measurements corresponding to angular velocity around the three sensor axes.

For the selected gyroscope range, the raw measurements are converted into angular velocity according to

$$\omega = \frac{G_{\text{raw}}}{S} \tag{1}$$

where G_{raw} represents the raw gyroscope measurement and S is the gyroscope scale factor.

For the $\pm 250^\circ/\text{s}$ configuration used in the project,

$$S = 131 \text{ LSB}/(^\circ/\text{s}) \tag{2}$$

The resulting angular velocities are transmitted as three comma-separated values.

3.2 Serial Communication

The Arduino sends the gyroscope measurements to Processing using a serial connection at 115200 baud.

Each transmitted line has the structure:

```
gyroX,gyroY,gyroZ
```

Processing waits for a complete line of serial data, separates the three values, and converts them into floating-point angular velocity measurements.

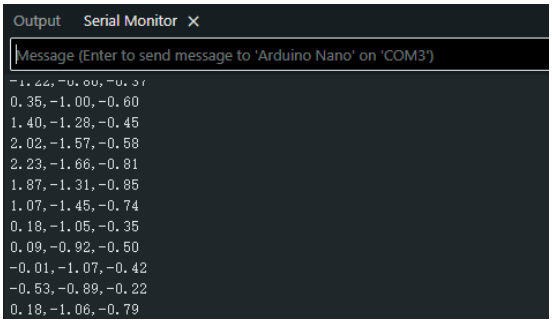

Figure 3: Serial data from MPU6050 through Arduino.

3.3 Initial 3D Rotation Prototype

Before implementing camera navigation, a simple 3D object was used to verify the complete communication pipeline between the MPU6050, Arduino, and Processing.

The measured angular velocities were integrated into approximate orientation angles and applied as rotations to a rendered cuboid in Processing.

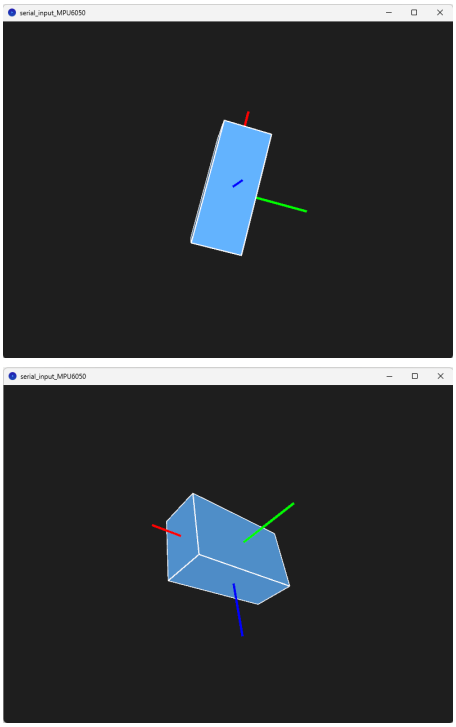

Figure 4: Initial Processing tests in which MPU6050 gyroscope motion controls the rotation of a 3D object.

From the rendered scene, it can be seen that the block object moves simultaneously while the object moves. user rotating the MPU6050 sensor connected with Arduino. This intermediate prototype made it possible to verify sensor input, serial communication, and graphical transformations before implementing camera control.

3.4 Orientation Estimation

The gyroscope measures angular velocity rather than orientation directly. Orientation is therefore estimated through numerical integration.

For yaw,

$$\psi_k = \psi_{k-1} + \omega_z \Delta t \tag{3}$$

and for pitch,

$$\theta_k = \theta_{k-1} + \omega_x \Delta t \tag{4}$$

Here, ψ represents yaw, θ represents pitch, and Δt is the elapsed time between consecutive sensor measurements.

3.5 Camera Transformation

The estimated yaw and pitch angles are converted into a viewing direction for the virtual camera.

The viewing direction can be represented as:

$$d_x = \sin(\psi)\cos(\theta) \tag{5}$$
$$d_y = \sin(\theta) \tag{6}$$
$$d_z = -\cos(\psi)\cos(\theta) \tag{7}$$

The resulting direction vector is used by Processing to define the camera target position.

This allows left-right sensor rotation to control camera yaw and up-down rotation to control camera pitch.

3.6 Panoramic Virtual Environment

A panoramic environment captured by iPhone app "360 Photo Cam" [2] is used as the virtual scene for evaluating camera control.

Figure 5: Panoramic picture captured using phone app.

The panoramic image is mapped onto the internal surface of a large sphere. The virtual camera is positioned near the center of the sphere, allowing camera rotation to simulate looking around inside the environment.

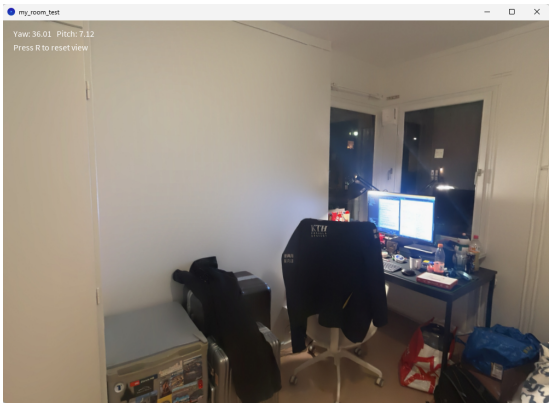

Figure 6: Panoramic virtual environment rendered in Processing.

This approach provides a lightweight virtual environment of a indoor room setup and avoids the additional complexity of constructing a complete 3D scene to validate the system.

3.7 Noise Reduction and Camera Reset

During testing, small non-zero gyroscope values were observed even when the MPU6050 sensor was kept stationary. Since the virtual camera angle is calculated by integrating angular velocity over time, these small errors can accumulate and cause visible camera drift. This affects the user experience because the view may slowly move even when the user is not intentionally rotating the sensor.

To reduce this unwanted movement, a deadband threshold is applied to the gyroscope measurements before updating the camera orientation. The filtered angular velocity is defined as

$$\omega_f = \begin{cases} 0 & \text{if } |\omega| < \omega_{\text{th}}, \\ omega, & \text{if } |\omega| \geq \omega_{\text{th}} \end{cases} \tag{8}$$

Here, ω is the measured angular velocity, ω_f is the filtered angular velocity, and ω_{th} is the deadband threshold. In the final implementation, the threshold was set to

$$\omega_{\text{th}} = 1.0^\circ/\text{s} \tag{9}$$

This means that angular velocity values smaller than $1.0^\circ/\text{s}$ are treated as stationary noise and set to zero. The purpose of this threshold is to prevent small sensor bias from being integrated into a large change in camera yaw or pitch.

In addition to the deadband method, Exponential Moving Average (EMA) smoothing was also tested. The smoothing equation is

$$\omega_s[n] = \alpha\omega_f[n] + (1-\alpha)\omega_s[n-1] \tag{10}$$

where $\omega_s[n]$ is the smoothed angular velocity at the current time step, $\omega_f[n]$ is the deadband-filtered angular velocity, and α is the smoothing factor. In the final test, the smoothing factor was set to

$$\alpha = 0.15 \tag{11}$$

The EMA filter is expected to make the rendered camera movement visually smoother. However, smoothing also introduces a trade-off because stronger smoothing can make the camera feel slightly delayed during quick motion.

A manual camera reset function is also implemented. When the reset key is pressed, the estimated yaw and pitch angles are set back to zero. This is important because the MPU6050 gyroscope does not provide an absolute yaw reference, so accumulated drift cannot be completely removed during long interaction periods.

4 Evaluation Method

Based on the deadband setup and filtering method, the prototype is evaluated using three sensor-processing configurations:

- **Raw gyroscope**: the gyroscope values are directly integrated into the camera yaw and pitch angles without additional filtering.
- **Deadband**: small gyroscope values below the selected threshold are removed before integration.
- **Deadband + EMA smoothing**: the deadband filter is first applied, followed by exponential moving average smoothing.

These three configurations allow the effect of each processing step to be compared. The raw gyroscope configuration is used as the baseline. The deadband configuration tests whether stationary drift can be reduced. The deadband with EMA smoothing configuration tests whether the camera motion can be made smoother while still remaining stable.

4.1 Stationary Drift Test

The main quantitative evaluation is the stationary drift test. In this test, the MPU6050 sensor is placed in a fixed position and the virtual camera is reset to zero yaw and zero pitch. The sensor is then kept still for 30 seconds. At the end of the test, the accumulated yaw and pitch deviation are recorded from the Processing console.

This test measures how much the virtual camera moves when there is no intentional user input. Ideally, both yaw and pitch drift should remain close to zero. A large drift value indicates that small gyroscope errors are being accumulated over time.

4.2 Responsiveness and Smoothness

Responsiveness is evaluated qualitatively by rotating the sensor by hand and observing how quickly the virtual camera responds. Smoothness is evaluated by observing whether the camera motion appears visually stable or jittery during sensor movement.

The current prototype does not use an external motion-capture system or high-speed camera, so the response delay is not measured numerically. Instead, the comparison focuses on perceived interaction quality.

5 Results

The final parameter setup used for the evaluation is shown in Table 1.

Table 1: Drift test parameters setup.

Parameter	Value
Test duration	30 s
Deadband threshold	$1.0^\circ/\text{s}$
EMA factor	$\alpha = 0.15$
Reset function	Manual yaw and pitch reset
Camera mapping	Gyro Z to yaw, gyro X to pitch

The stationary drift test results printed from the Processing console are shown in Table 2. The results show the accumulated yaw and pitch drift after 30 seconds.

Table 2: Stationary drift test results over a 30-second interval.

Method	Yaw Drift [°]	Pitch Drift [°]
Raw gyro	16.19	20.50
Deadband	0.00	0.03
Deadband + EMA	0.00	0.06

The raw gyroscope configuration produced significant drift. After 30 seconds, the virtual camera drifted by 16.19° in yaw and 20.50° in pitch. This confirms that directly integrating raw gyroscope data is not stable enough for long-duration camera control.

After applying the deadband threshold, the stationary drift was greatly reduced. The yaw drift became 0.00° and the pitch drift was only 0.03°. This result shows that the main source of stationary drift was small non-zero gyroscope values when the sensor was at rest. By removing these values before integration, the virtual camera remained almost stationary during the test.

The deadband with EMA smoothing configuration produced a similar result, with 0.00° yaw drift and 0.06° pitch drift. This indicates that the deadband filter was the most important part for reducing stationary drift. EMA smoothing did not significantly improve the stationary drift result, but it can still improve the perceived smoothness of camera motion during active movement.

6 Discussion

6.1 Filtering Performance

The three tested configurations demonstrate the trade-off between responsiveness and stability. The raw gyroscope setup provides the most direct response because the sensor values are integrated without additional processing. However, the stationary drift test shows that this method is not stable enough for continuous camera control. Even small non-zero gyroscope values accumulate over time and cause the virtual view to move when the sensor is stationary.

Applying a deadband threshold significantly improves stability. With $\omega_{\text{th}} = 1.0^\circ/\text{s}$, small unwanted gyroscope values are removed before integration. This reduced the 30-second stationary drift from 16.19° yaw and 20.50° pitch in the raw setup to almost zero drift. This makes the camera view more controllable when the user is trying to keep the viewpoint fixed.

The deadband with EMA smoothing produced a similar stationary result. In this test, smoothing did not further reduce drift significantly, because the deadband already removed most stationary noise. Its main benefit is instead to reduce visual jitter during active movement. However, excessive smoothing can also make the camera feel delayed, so the smoothing factor must be kept moderate.

6.2 User Experience

The filtering and reset functions improve the interaction experience by making the virtual camera more stable and predictable. Without filtering, the camera slowly drifts even when the user is still, which makes it difficult to focus on a fixed point in the panoramic scene. The deadband filter reduces this problem and gives the user better control of the viewpoint, meanwhile it reduces the chance of getting 3-D sickness.

The EMA filter can make the motion visually smoother during rotation, which is useful for panoramic viewing and game-like interaction. The manual reset function is also important because it allows the user to quickly re-center the view if drift accumulates during longer use. Together, these additions make the system more suitable as an interactive prototype rather than only a sensor-reading demonstration.

6.3 Limitations

The current prototype is still limited by gyroscope drift. Since the camera orientation is estimated by integrating angular velocity, any remaining sensor bias can accumulate over time. This is especially important for yaw, because the MPU6050 does not provide an absolute heading reference.

The selected deadband threshold also introduces a trade-off. A larger threshold improves stationary stability, but very slow intentional motion may be ignored. The responsiveness evaluation is qualitative, since no external motion-capture or latency measurement system was used. In addition, the virtual environment is based on a recorded panoramic image, so it does not include true 3D depth or physical interaction with scene geometry.

6.4 Future Improvements

Future work could first improve the sensor processing. A calibration step could be added to estimate and subtract the stationary gyroscope bias before integration. A complementary filter or more advanced sensor-fusion method could also be implemented by combining gyroscope and accelerometer data.

The interaction design could also be extended. For example, an escape-room style system could be added, where looking at specific objects in the panorama triggers images, animations, or clues. This would make the prototype more game-like and better connected to interaction and rendering.

7 Conclusion

This project implemented a low-cost IMU-based camera control prototype using an MPU6050 sensor, an Arduino Nano, and Processing. Gyroscope data is transmitted through serial communication and mapped to virtual camera yaw and pitch in a panoramic scene.

The prototype demonstrates that physical sensor motion can be used as an input method for controlling a rendered viewpoint. The evaluation shows that raw gyroscope integration causes significant drift, with 16.19° yaw drift and 20.50° pitch drift after 30 seconds. After applying a 1.0°/s deadband threshold, the drift was reduced to 0.00° in yaw and 0.03° in pitch. Adding EMA smoothing produced a similar stationary result, with 0.00° yaw drift and 0.06° pitch drift.

These results show that deadband filtering is effective for improving stationary stability in the current prototype. EMA smoothing can improve perceived motion smoothness, while the manual reset function provides a simple way to recover from accumulated drift. Overall, the MPU6050 and Arduino Nano provided sufficiently responsive input for a real-time camera-control demonstration, although the system remains limited by gyroscope drift, lack of absolute yaw reference, and the simplified panoramic environment.

References

[1] Tobii Gaming, “Euro Truck Simulator 2 - Eye and Head Tracking,” https://gaming.tobii.com/games/euro-truck-simulator-2/, accessed: Apr. 20, 2026.

[2] DoSpace, Inc., “360 Photo Cam: Best 360 Camera App & Panorama App,” https://360photocam.com/, 2025, accessed: Aug. 30, 2026.